SIMATS ENGINEERING

ASSESSMENT TOOL 3 – CASE STUDY

Course Name	Internet Programming
Course Code	CSA4304
Course Outcome	CO-1
Assessment Tool	Application-Based Questions
Total Marks	40%

Name / Reg No: P.Revanth Reddy

Question 1: Select the correct HTML tag for video embedding and justify your answer.

Answer: Option B – <video src="video.mp4" controls></video>

Justification:

- <video> is the standard HTML5 element for embedding video directly in a page. It plays the file in the browser without any plug-in such as Flash.
- The **src** attribute gives the file to play, and the **controls** attribute shows the built-in play, pause, volume, seek and full-screen buttons, so the visitor can control the promotional video.
- Text placed between the opening and closing tags is shown as fallback content in browsers that do not support the element.
- It works with the HTML5 attributes poster, autoplay, muted, loop and preload, and with <track> for captions, which supports accessibility.

Why the other options are wrong:

Option	Tag	Reason
A	<media>	Not a valid HTML element; the browser treats it as an unknown tag and shows no player.
C	<movie>	Not a valid HTML element; it is not defined in any HTML standard.
D	<vid>	Not a valid HTML element; it is only an invented abbreviation.

Recommended implementation:

```
<video width="640" height="360" controls poster="thumb.jpg" preload="metadata">
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  Your browser does not support the video tag.
</video>
```

Providing MP4 and WebM sources improves compatibility, and the poster image and preload="metadata" make the homepage load faster.

Question 2: Identify the appropriate HTTP method for form submission and explain why.

Answer: Option B – POST

Explanation:

- POST sends the form data inside the **body** of the HTTP request, so it does not appear in the URL, browser history, bookmarks or server access logs. This is essential for a registration form that carries passwords and personal details.
- POST has no practical limit on data size, whereas GET is limited by URL length.
- POST is meant for submitting data that creates or changes something on the server, such as a new user account. It is not cached and is not repeated silently by the browser.
- POST supports file uploads through enctype="multipart/form-data".

POST hides the data from the URL but does not encrypt it, so the form must also be served over **HTTPS**, and the server should hash passwords and validate input.

Why the other methods are not suitable:

Option	Reason
A. GET	Puts the data in the URL (for example /register?user=ali&pass=1234), so credentials are visible and stored in history and logs. GET is meant only for retrieving data.
C. PUT	Replaces or updates an existing resource at a known URL and is used in REST APIs. HTML forms cannot submit with PUT, and it is not meant for creating a registration.

Option	Reason
D. DELETE	Removes a resource. It is the opposite of what a registration form should do, and HTML forms do not support it.

Recommended implementation:

```
<form action="/register" method="post">
  <input type="text" name="username" required>
  <input type="email" name="email" required>
  <input type="password" name="password" minlength="8" required>
  <button type="submit">Register</button>
</form>
```

Question 3: Choose the proper semantic element for navigation and justify its usage.

Answer: Option B – <nav>

Justification:

- <nav> is the HTML5 semantic element that marks a block of **navigation links** such as the main menu, a table of contents or pagination. Its name states its purpose, so the code is self-describing.
- **Accessibility:** screen readers announce it as a "navigation" landmark, so users can jump straight to the menu or skip it.
- **SEO:** search engines understand which links form the site structure.
- **Maintainability:** developers and CSS can find and style the menu easily, and the page follows web standards.

Why the other options are not proper:

Option	Reason
A.	A generic container with no meaning. It can hold a menu, but it tells browsers and assistive technology nothing.
C. <section>	Groups related content under a heading (a thematic section), not a set of navigation links.
D. <header>	Holds introductory content such as a logo or title. It may contain a <nav>, but it is not the navigation element itself.

Recommended implementation:

```
<header>
 <nav aria-label="Main menu">
  <ul>
   <li><a href="index.html">Home</a></li>
   <li><a href="about.html">About</a></li>
   <li><a href="contact.html">Contact</a></li>
  </ul>
 </nav>
</header>
```

Question 4: Identify the correct XML syntax and explain the error in other options.

Answer: Option B – <note><to>User</to><from>Admin</from></note>

Explanation: Option B is **well-formed** XML. It has a single root element (<note>), every start tag has a matching end tag, and the elements are properly nested, with <to> and <from> opened and closed inside <note>. XML is strict and case-sensitive, and a document that breaks these rules is rejected by the parser.

Errors in the other options:

Option	Code	Error
A	<note><to>User</to><from>Admin</from>	The root element <note> is never closed; the end tag </note> is missing.
C	<note><to>User</from></note>	The start tag <to> is closed by </from>, which does not match, so the nesting is improper. There is also no opening <from> tag.
D	<note to="User"></note	The end tag is incomplete: the closing ">" is missing in </note. (The attribute syntax to="User" itself is valid.)

Rules of well-formed XML applied:

- Exactly one root element.
- Every start tag has a matching, correctly spelled end tag.
- Elements are properly nested (the last element opened is the first closed).
- Tag names are case-sensitive and attribute values are quoted.

A complete configuration file following these rules:

```
<?xml version="1.0" encoding="UTF-8"?>
<note>
  <to>User</to>
  <from>Admin</from>
</note>
```

Question 5: Choose the correct protocol for web communication and justify your choice.

Answer: Option B – HTTP (in practice HTTPS)

Justification:

- HTTP (HyperText Transfer Protocol) is the application-layer protocol designed for the Web. A browser (client) sends a request and the web server returns a response containing HTML, CSS, images, JSON and other content.
- It runs over TCP, by default on port 80, and carries the methods (GET, POST), status codes (200, 404, 500) and headers used to deliver web pages and submit forms.
- For a registration form and user data the application should use **HTTPS** (HTTP over TLS, port 443). It encrypts the traffic and protects the confidentiality and integrity of the data.

Why the other protocols are not suitable:

Option	Protocol	Purpose (why it does not fit)
A	FTP (File Transfer Protocol, ports 20/21)	Transfers files between hosts; it does not deliver web pages to browsers.
C	SMTP (Simple Mail Transfer Protocol, port 25/587)	Sends e-mail between mail servers; not used for web pages.

Option	Protocol	Purpose (why it does not fit)
D	SNMP (Simple Network Management Protocol, port 161)	Monitors and manages network devices such as routers and switches.

Therefore the startup should deliver the web application over HTTP, secured as HTTPS.